# コア REST の実装

## Astra Automation

NetApp
May 03, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/astra-automation-2108/rest-core/rest_web_services.html on May 03, 2022. Always check docs.netapp.com for the latest.

# 目次

コア REST の実装 . . . . . 1
REST Web サービス . . . . . 1
リソースとコレクション . . . . . 2
HTTP の詳細 . . . . . 3
URL 形式 . . . . . 6

# コア REST の実装

## REST Web サービス

Representational State Transfer （ REST ）は、分散 Web アプリケーションの作成に使用される形式です。Web サービス API の設計に適用されることで、サーバベースのリソースを公開し、その状態を管理するための一連の主流テクノロジとベストプラクティスが確立されます。REST はアプリケーション開発のための一貫した基盤を提供しますが、各 API の詳細は設計内容に応じて異なる場合があります。ライブ環境で使用する前に、 Astra Control REST API の特性を理解しておく必要があります。

リソースと状態の表示

リソースは、 Web ベースシステムの基本コンポーネントです。REST Web サービスアプリケーションを作成する場合、設計の早い段階で次の作業を行います。

- システムまたはサーバベースのリソースの識別

  すべてのシステムは、リソースを使用および管理します。リソースには、ファイル、ビジネストランザクション、プロセス、管理エンティティなどがあります。REST Web サービスに基づいてアプリケーションを設計する際に行う最初の作業の 1 つは、リソースを識別することです。

- リソースの状態および関連する状態操作の定義

  リソースの状態の数は有限で、リソースは必ずそのいずれかの状態にあります。状態、および状態の変化に影響する関連操作を明確に定義する必要があります。

**URI** エンドポイント

すべての REST リソースは、明確に定義されたアドレス指定方式を使用して定義および使用可能にする必要があります。リソースが置かれているエンドポイントは、 Uniform Resource Identifier （ URI ）で識別されます。URI は、ネットワーク内の各リソースに一意の名前を作成するための一般的なフレームワークです。Uniform Resource Locator （ URL ）は、リソースを識別してアクセスするために Web サービスで使用される URI の一種です。リソースは通常、ファイルディレクトリに似た階層構造で公開されます。

**HTTP** メッセージ

Hypertext Transfer Protocol （ HTTP ）は、 Web サービスのクライアントとサーバがリソースに関する要求と応答のメッセージを交換する際に使用するプロトコルです。Web サービスアプリケーションの設計の一環として、 HTTP メソッドはリソースおよび対応する状態管理アクションにマッピングされます。HTTP はステートレスです。したがって、関連する一連の要求と応答を 1 つのトランザクションの一部として関連付けるには、要求と応答のデータフローで伝送される HTTP ヘッダーに追加情報を含める必要があります。

**JSON** 形式

Web サービスのクライアントとサーバの間で情報を構造化して転送する方法は複数ありますが、最も広く使用されているのは JavaScript Object Notation （ JSON ）です。JSON は、単純なデータ構造をプレーンテキストで表すための業界標準であり、リソースについての状態情報の転送に使用されます。Astra Control REST API では、 JSON を使用して、各 HTTP 要求と応答の本文で伝送されるデータをフォーマットします。

# リソースとコレクション

Astra Control REST API を使用すると、リソースインスタンスとリソースインスタンスのコレクションにアクセスできます。

> 概念的には REST * リソース * は、オブジェクト指向プログラミング（ OOP ）言語およびシステムで定義される * オブジェクト * に似ています。これらの用語が同じ意味で使用されることもあります。ただし一般には、外部 REST API のコンテキストで使用する場合は「リソース」が推奨され、サーバに格納される対応するステートフルインスタンスデータには「オブジェクト」が使用されます。

## Astra リソースの属性

Astra Control REST API は、 RESTful 設計の原則に準拠しています。各アストラリソースインスタンスは、明確に定義されたリソースタイプに基づいて作成されます。同じタイプのリソースインスタンスのセットを * 集合 * と呼びます。API 呼び出しは、個々のリソースまたはリソースの集合に対して機能します。

### リソースタイプ

Astra Control REST API に含まれるリソースタイプには、次の特徴があります。

- すべてのリソースタイプはスキーマを使用して定義されます（通常は JSON 形式）。
- すべてのリソーススキーマには、リソースタイプとバージョンが含まれています
- リソースタイプはグローバルに一意です

### リソースインスタンス

Astra Control REST API から使用できるリソースインスタンスには、次のような特徴があります。

- リソースインスタンスは、単一のリソースタイプに基づいて作成されます
- リソースタイプは、 ［ メディアタイプ ］ の値を使用して示されます
- インスタンスは、アストラサービスによって管理されるステートフルデータで構成されます
- 各インスタンスには、一意で長く存続する URL を使用してアクセスできます
- リソースインスタンスが複数のリプレゼンテーションを持つことができる場合は、異なるメディアタイプを使用して必要なリプレゼンテーションを要求できます

### リソースコレクション

Astra Control REST API で使用できるリソース収集には、次のような特徴があります。

- 単一のリソースタイプの一連のリソースインスタンスをコレクションと呼びます
- 一連のリソースには、一意で一時的な URL があります

### インスタンス **ID**

すべてのリソースインスタンスには、作成時に識別子が割り当てられます。この識別子は 128 ビットの UUIDv4 値です。割り当てられた UUIDv4 の値は、グローバルに一意で変更できません。新しいインスタンス

を作成する API 呼び出しを発行すると、関連付けられた ID を持つ URL が HTTP 応答の「 Location 」ヘッダーで呼び出し元に返されます。リソースインスタンスを以降の呼び出しで参照する際には、この識別子を抽出して使用できます。

> リソース識別子は、コレクションで使用される主キーです。

## Astra のリソースに共通の構造

すべてのアストラ制御リソースは、共通の構造を使用して定義されます。

共通のデータ

すべてのアストラリソースには、次の表に示すキーと値が含まれています。

| キーを押します | 説明 |
|---|---|
| を入力します | グローバルに一意なリソースタイプ。 * リソースタイプ * と呼ばれます。 |
| バージョン | * リソースバージョン * と呼ばれるバージョン識別子。 |
| ID | グローバル一意識別子。 * リソース識別子 * と呼ばれます。 |
| メタデータ | ユーザラベルやシステムラベルなどのさまざまな情報を含む JSON オブジェクト。 |

メタデータオブジェクト

各アストラリソースに含まれるメタデータ JSON オブジェクトには、次の表に示すキーと値が含まれています。

| キーを押します | 説明 |
|---|---|
| ラベル | リソースに関連付けられているクライアント指定のラベルの JSON 配列。 |
| 作成タイムスタンプ | リソースが作成されたときのタイムスタンプを含む JSON 文字列。 |
| modificationTimestamp | リソースの最終変更日時を示す ISO-8601 形式のタイムスタンプを含む JSON 文字列。 |
| 作成者 | リソースを作成したユーザ ID の UUIDv4 識別子を含む JSON 文字列。リソースが内部システムコンポーネントによって作成され、作成するエンティティに関連付けられている UUID がない場合は、「 * null * UUID 」が使用されます。 |

リソースの状態

選択されたリソースは ' ライフサイクルの移行とアクセス制御のオーケストレーションに使用される価値を示します

# HTTP の詳細

Astra Control REST API は、 HTTP および関連パラメータを使用して、リソースとコレクションに対して操作を行います。HTTP 実装の詳細を以下に示します。

## API トランザクションと CRUD モデル

Astra Control REST API は、明確に定義された操作と状態の移行を伴うトランザクションモデルを実装します。

### 要求と応答の API トランザクション

すべての REST API 呼び出しは、 Astra サービスへの HTTP 要求として実行されます。要求ごとに、関連付けられた応答がクライアントに生成されます。この要求と応答のペアで API トランザクションを使用できます。

### CRUD 操作モデルのサポート

Astra Control REST API で使用できる各リソースインスタンスとコレクションには、 *CRUD * モデルに基づいてアクセスします。4 つの操作があり、それぞれが単一の HTTP メソッドにマッピングされます。処理は次のとおりです。

- 作成
- 読み取り
- 更新
- 削除

一部の Astra リソースでは、これらの操作の一部のみがサポートされています。を確認しておきます "API リファレンス" 特定の API 呼び出しに関する詳細については、を参照してください。

## HTTP メソッド

次の表に、 API でサポートされる HTTP メソッドまたは動詞を示します。

| メソッド | CRUD | 説明 |
|---|---|---|
| 取得 | 読み取り | リソースインスタンスまたはコレクションのオブジェクトプロパティを取得します。これは、コレクションとともに使用される場合、 * list * 演算と見なされます。 |
| 投稿（ Post ） | 作成 | 入力パラメータに基づいて新しいリソースインスタンスを作成します。長期 URL は 'Location' 応答ヘッダーで返されます |
| PUT | 更新 | 指定した JSON 要求の本文でリソースインスタンス全体を更新します。ユーザが変更できないキー値は保持されます。 |
| 削除 | 削除 | 既存のリソースインスタンスを削除します。 |

### 要求と応答のヘッダー

次の表に、 Astra Control REST API で使用される HTTP ヘッダーを示します。

を参照してください "RFC 7232" および "RFC 7233" を参照してください。

| ヘッダー | を入力します | 使用上の注意 |
| --- | --- | --- |
| 同意します | リクエスト | 値が「 * / * 」または指定されていない場合、「 application/json 」は Content-Type 応答ヘッダーで返されます。値が Astra リソースのメディアタイプに設定されている場合、同じメディアタイプが Content-Type ヘッダーに返されます。 |
| 承認 | リクエスト | ユーザーの API キーを使用した Bearer トークン。 |
| コンテンツタイプ | 応答 | 「 Accept 」要求ヘッダーに基づいて返されます。 |
| ETag | 応答 | RFC 7232 で定義されたとおりに、成功したに含まれます。値は、 JSON リソース全体の MD5 値を 16 進数で表したものです。 |
| if-match | リクエスト | セクション 3.1 「 RFC 7232 」で説明されているように、 * PUT * 要求をサポートする前提条件の要求ヘッダーが実装されています。 |
| If-Modified-Since の略 | リクエスト | セクション 3.4 「 RFC 7232 」に記載されている前提条件の要求ヘッダーと、 *PUT * 要求のサポート。 |
| If - unmodified- since | リクエスト | セクション 3.4 「 RFC 7232 」に記載されている前提条件の要求ヘッダーと、 *PUT * 要求のサポート。 |
| 場所 | 応答 | 新しく作成されたリソースの完全な URL が含まれます。 |

クエリパラメータ

リソースコレクションでは、次のクエリパラメータを使用できます。を参照してください "コレクションの操作" を参照してください。

| クエリパラメータ | 説明 |
| --- | --- |
| 含める | コレクションを読み取るときに返すフィールドが含まれています。 |
| フィルタ | コレクションの読み取り時にリソースが返されるために一致しなければならないフィールドを示します。 |
| OrderBy | コレクションの読み取り時に返されるリソースのソート順序を指定します。 |
| 制限（ Limit ） | コレクションの読み取り時に返されるリソースの最大数を制限します。 |
| スキップします | コレクションの読み取り時に、パスオーバーおよびスキップするリソースの数を設定します。 |
| カウント | メタデータオブジェクトで返されるリソースの総数。 |

**HTTP** ステータスコード

Astra Control REST API で使用される HTTP ステータスコードを次に示します。

> Astra Control REST API は、 HTTP API * 標準の * 問題の詳細も使用します。を参照してください "診断とサポート" を参照してください。

| コード | 意味 | 説明 |
|---|---|---|
| 200 | わかりました | 新しいリソースインスタンスを作成しない呼び出しが成功したことを示します。 |
| 201 | 作成済み | オブジェクトが作成され、場所の応答ヘッダーにオブジェクトの一意の識別子が含まれている。 |
| 204 | コンテンツがありません | 要求は成功しましたが、コンテンツが返されませんでした。 |
| 400 | 無効な要求です | 要求の入力が認識されないか不適切です。 |
| 401 | 権限がありません | ユーザは認証されていないため、認証が必要です。 |
| 403 | 禁止されている | 認証エラーによりアクセスが拒否されました。 |
| 404 | が見つかりません | 要求で参照されているリソースが存在しません。 |
| 409 | 競合しています | オブジェクトがすでに存在するため、オブジェクトの作成に失敗しました。 |
| 500 | 内部エラー | サーバで一般的な内部エラーが発生しました。 |
| 503 | サービスを利用できません | サービスは何らかの理由で要求を処理する準備ができていません。 |

# URL 形式

REST API を使用したリソースインスタンスまたはコレクションへのアクセスに使用される URL の一般的な構造は、いくつかの値で構成されます。この構造は、基礎となるオブジェクトモデルとシステム設計を反映しています。

## URL の形式

URL の形式と例を次に示します。

### root としてアカウントを作成します

すべての REST エンドポイントへのリソースパスのルートは Astra アカウントです。そして、 URL 内のすべてのパスは「 /account/{account_id} 」で始まります。ここで、「 account_id 」はアカウントの一意の UUIDv4 値です。内部構造：すべてのリソースアクセスが特定のアカウントに基づいている設計を反映します。

エンドポイントリソースカテゴリ

Astra リソースエンドポイントは、次の 3 つのカテゴリに分類されます。

- コア (`/core')
- 管理対象アプリケーション (`/k8s')
- トポロジ (`/topology')

を参照してください "リソース" を参照してください。

カテゴリバージョン

3 つのリソースカテゴリのそれぞれに、アクセスされるリソースのバージョンを制御するグローバルバージョンがあります。規則と定義により ' リソース・カテゴリの新しいメジャー・バージョン (/v1' から /v2') に移行すると 'API の変更が破壊されます

リソースインスタンスまたはコレクション

リソースインスタンスまたはコレクションにアクセスするかどうかに基づいて、パスでリソースタイプと識別子の組み合わせを使用できます。

例

- リソースパス

  上記の構造に基づいて、エンドポイントへの一般的なパスは、「 /accounts/{account_id}/core/v1/users` です。

- 完全な URL

  対応するエンドポイントの完全な URL は、 https://astra.netapp.io/accounts/{account_id}/core/v1/users` です。

**Copyright Information**

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

**Trademark Information**

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.